

# What the BIOS Cannot See

Missing thermal paste, a pump spinning free, a switched-off radiator: the BIOS throttles, then stays silent. Sensor physics, correlated by deterministic rules, names the fault — and the agent explains. A frugal architecture: no resident intelligence, zero bytes in the flash, the firmware remains the safeguard.

**Ryzen 9 5950X · AIO 240 · ASUS B450**

omarchy-firmware Phase 2 · fw.diag.thermal · 82 checks · R\_th · baseline

# Contents

|                                                                                 |           |
|---------------------------------------------------------------------------------|-----------|
| <b>1. The case the BIOS cannot name</b>                                         | <b>0</b>  |
| 1.1 The chain of silence: what the BIOS actually does . . . . .                 | 1         |
| 1.2 The plan of this volume . . . . .                                           | 1         |
| <b>2. Taxonomy of the faults invisible to the BIOS</b>                          | <b>2</b>  |
| 2.1 Slow degradation, the absolute blind spot . . . . .                         | 3         |
| <b>3. The physics of diagnosis: R_th and signatures</b>                         | <b>3</b>  |
| 3.1 The decision tree: from symptom to named verdict . . . . .                  | 4         |
| 3.2 Reading the steady state, naming the power fold-back . . . . .              | 5         |
| <b>4. The frugal architecture: event-driven intelligence</b>                    | <b>6</b>  |
| 4.1 The budget, quantified without complacency . . . . .                        | 7         |
| 4.2 Why there will never be AI in the BIOS . . . . .                            | 8         |
| 4.3 The active probe: heating deliberately, but bounded and announced . . . . . | 8         |
| <b>5. Proof by code: Phase 2 of omarchy-firmware</b>                            | <b>9</b>  |
| 5.1 The memory the BIOS does not have . . . . .                                 | 9         |
| 5.2 The human keeps control, at every level . . . . .                           | 10        |
| <b>6. The limits, stated frankly</b>                                            | <b>10</b> |
| <b>7. Roadmap and overall coherence</b>                                         | <b>12</b> |
| 7.1 What the agent will never do — and why that is the good news . . . . .      | 12        |
| 7.2 What this volume adds to the whole . . . . .                                | 12        |
| <b>Sources and references</b>                                                   | <b>13</b> |

## 1. The case the BIOS cannot name

It all starts with a mundane example that remains unsolvable for today's computing: a user builds a machine — a Ryzen 9 5950X cooled by a pump cooler, on an ASUS B450 board — and forgets the thermal paste, or applies so little of it that contact exists only at a few points. The machine boots, the POST displays, Linux starts: nothing signals the anomaly. Then the load arrives, the processor temperature jumps, and the BIOS applies the only response it knows: reduce frequency at Tjmax (90 °C on a 5950X), hold, then shut down if the curve does not come back down. The machine is overheating; the user changes the fan curve, retests, replaces the monitoring software, ends up dismantling the whole case — and the complete diagnosis will still fit in three words: no paste. This volume starts from that scene, posed as a working question: **is it detectable, and who should detect it?**

The answer fits in one sentence: it is detectable — chapter 3 proves it with the most elementary physics — and the detector can be neither the BIOS nor the user: it is the agent. The BIOS cannot detect it because it has neither the view nor the model: it reads one temperature (Tctl), it is blind to the coolant temperature, to the power actually dissipated, to the machine's history, and it has no representation of the cooling chain that would let it conclude the die-to-coolant path is broken while the coolant itself stays cold. The user cannot

detect it without opening the machine, because the clues are scattered across sensors no consumer software correlates. The agent, meanwhile, gathers exactly what the other two lack: access to every sensor the kernel exposes, a physical model of cooling, long-term memory, and the ability to explain its verdict in human language.

|                                                                                                                |                                                                                   |                                                                                      |                                               |
|----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|-----------------------------------------------|
| <b>0.29</b><br>$^{\circ}\text{C}/\text{W}$ — healthy $R_{\text{th}}$ of a 5950X on a 240 mm AIO (steady state) | <b>0.585</b><br>$^{\circ}\text{C}/\text{W}$ — the same CPU, thermal paste missing | <b>4.0 s</b><br>to reach $85^{\circ}\text{C}$ — against 12-20 s with a healthy joint | <b>0</b><br>bytes added to the BIOS SPI flash |
|----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|-----------------------------------------------|

## 1.1 The chain of silence: what the BIOS actually does

Credit must be given where due: in the scenario above, the BIOS works perfectly. As soon as  $T_{\text{ctl}}$  reaches  $T_{\text{jmax}}$ , AGESA folds the consumed power — Package Power Tracking drops from 142 W to under 100 W — frequencies fall, and if the temperature persists, the hardware shutdown fires. It is a remarkably reliable survival chain, tested on millions of machines, and this volume never intended to replace it: within the proposed architecture it remains the final, untouchable safeguard. The problem is not what the BIOS does, it is what it does not say: the protection applies in silence, with no named cause, no distinction between a dead pump, a clogged radiator, missing paste or simply insufficient airflow. Four radically different faults produce the same symptom — and therefore four different repairs for a single message: the CPU got hot.

This blindness is not vendor laziness: it is structural. The BIOS runs in a pre-OS environment, with no file system and no network, on a 16 to 32 MB SPI flash already saturated by AGESA, and it only has the sensors wired to its logic —  $T_{\text{ctl}}$ , a few board thermistors, the fan headers. It has nowhere to put a thermal memory, no room for a cooling model, and no interface to explain anything. Add the constraint this study set from the start: **the firmware must not become a system hungrier than the base BIOS** — any idea of embedded intelligence in the BIOS is thus eliminated on principle, before the technical question even arises. Chapter 4 will show that this constraint is not a limitation: it is what forces the right architecture.

## 1.2 The plan of this volume

Chapter 2 draws up the taxonomy of the faults the BIOS does not see — from brutal physical failures to slow degradations, including the stage throttles the hardware endures without naming them. Chapter 3 builds the physics of diagnosis: the thermal network, the  $R_{\text{th}}$  resistance, time constants, and the decision tree that turns scattered sensors into named verdicts. Chapter 4 sets the frugal three-layer architecture, where intelligence is event-driven and never resident. Chapter 5 documents what was actually built — Phase 2 of omarchy-firmware, its fifth T0 tool, its eight test scenarios and its 82 checks. Chapter 6 establishes the honest list of limits, and chapter 7 places the whole back into the trajectory of the previous volumes. A convention is kept from volumes 1 and 2: everything asserted here has been executed, measured or tested on the base delivered with this report. This English edition adds the epilogue the French one could not have: the

signature surface has since grown from 8 to 12 scenarios, the suite now passes 281 checks, and the whole behavioural contract is rehearsed nightly against a digital twin.

## 2. Taxonomy of the faults invisible to the BIOS

The BIOS errors a user encounters fall into four families, which the previous volumes had already crossed. The first is the **firmware bug** — stuttering fTPM, LogoFAIL, Sinkclose: repairable by an AGESA update, that is fw.audit.cve's territory. The second is the **configuration mistake** — XMP disabled, an unsuitable boot mode, a wild Q-Fan curve: repairable by better settings, the future T1 tools' territory. The third, at the heart of this volume, is the **physical failure** — paste, pump, fan, airflow, power: repairable only in the real world, by hand, and for that it must first be named. The fourth is **slow degradation** — accumulated dust, drying paste, ageing capacitors: it does not even exist at the level of a single boot; it only appears over time. The BIOS fails on the last two for the same root reason: it has neither the sensors, nor the model, nor the memory.

| Physical failure                                | What the BIOS does                          | What the agent sees (correlated sensors)                                                     | Named finding                                                    |
|-------------------------------------------------|---------------------------------------------|----------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| Thermal paste missing, dried, or loose mounting | Throttle at Tjmax, then shutdown — no cause | die→coolant R_th > 0.42 °C/W with a cold coolant; 85 °C reached in under 8 s                 | interface-degradee                                               |
| AIO pump dead or unplugged                      | Rapid shutdown at first load                | Pump header at 0 rpm, Tctl high from idle, radiator fans spinning for nothing                | pompe-morte                                                      |
| Dead radiator fan                               | Progressive throttle under load             | coolant – ambient > 15 °C while R_th stays healthy (≤ 0.32): the die→coolant path is fine    | liquide-chaud, ventilateur-zero-tr                               |
| Dust (filters, radiator) — slow degradation     | Nothing: every boot looks normal            | Drift of R_th or of the idle ΔT over weeks (reference baseline compared)                     | degradation-progress<br>ive                                      |
| VRM (power stages) overheating                  | The CPU lowers its clocks, unlabelled       | VRM sensor > 89 °C while Tctl stays moderate — the stage throttle is invisible to the BIOS   | vrn-chaud                                                        |
| +12 V rail sagging under load                   | Freezes, random resets, sometimes nothing   | Super I/O ADC reading < 11.4 V under load (± 3 % accuracy, weak but real signal)             | tension-12v-basse                                                |
| NVMe, chipset or GPU hot spot                   | Nothing (outside the CPU)                   | Generic hwmon sensors > 85 °C, correlated with disk or graphics use                          | capteur-chaud                                                    |
| Global cooling insufficiency                    | High thermal plateau, repeated throttling   | Plateau ≥ 88 °C under controlled load, measured power fold-back (140 → 96 W in the scenario) | refroidissement-insu<br>ffisant, protection-th<br>ermique-active |

Table 2.1 — Eight faults the BIOS conflates, and what sensor correlation says about them.

Two remarks on reading this table. First, the remedies differ radically from row to row — refill a joint, replace a pump, dust out, rework the airflow, check the power supply — while the BIOS symptom is identical: that invisible difference is precisely what costs the user hours and the manufacturers support

returns. Second, none of these findings requires exotic hardware: every sensor involved already exists on a standard B450/B550 — k10temp for the CPU, the Nuvoton Super I/O for the headers, the board thermistors, the AIO's coolant sensor when present. The rarity is not in the hardware; it is in the correlation.

## 2.1 Slow degradation, the absolute blind spot

The fourth family deserves a development of its own, because it is the most insidious. A machine accumulating dust loses cooling efficiency over weeks:  $R_{th}$  drifts, say, from 0.24 to 0.34 °C/W without any single snapshot ever being alarming — the plateau stays below the thresholds, the BIOS never throttles, the user only sees a somewhat noisy machine. No existing system detects this, for one simple reason: it would require remembering the measurements of six months ago, and the BIOS has neither a reliable clock to date with nor safe storage to archive in, while classic monitoring software only displays the present instant. Phase 2's base introduces exactly that missing memory: a *baseline* written to the XDG user state, dated, replayable, which every diagnosis compares against its predecessor — with a drift threshold of 25 percent over at least seven days to keep the noise out. To our knowledge, this is the first time a desktop AM4 suite explicitly measures that cooling is degrading instead of noting that it is hot.

*There are errors in the BIOS that are often very hard to resolve — errors you simply cannot fix. For example the CPU overheats because the person did not apply thermal paste: is that detectable? The person has a pump cooler but the CPU really overheats. There are things the BIOS today cannot solve, but that we can solve as AI advances. — the founding question of this volume, and the answer, chapter after chapter: yes, detectable, nameable, explainable — provided you do not put it inside the BIOS.*

## 3. The physics of diagnosis: $R_{th}$ and signatures

---

A processor's cooling is a chain of successive conductions: the silicon die transfers its heat to the integrated heat spreader (IHS), the IHS passes it through the thermal joint to the cooler's cold plate, the liquid carries it to the radiator, and the radiator gives it up to the ambient air. Each link opposes a thermal resistance, and their sum governs the gap between the die temperature and room temperature. The usable quantity is computed at steady state:  $R_{th} = (T_{die} - T_{coolant}) / P_{package}$ , in °C/W. On a 5950X well cooled by a 240 mm AIO, with a 142 W PPT, the healthy order of magnitude sits between 0.26 and 0.32 °C/W; missing paste or a loose mount pushes the joint resistance alone to values the sum can no longer hide. The beauty of this quantity is that it is deconditioned from the environment: it absorbs room temperature, the exact load of the moment, the fan curve — and leaves only the state of the contact visible.

Second signature: the **time constant**. A healthy joint damps the ramp to load — on a 5950X, reaching 85 °C from a stable idle typically takes 12 to 20 seconds, the time the thermal mass of the IHS and the block needs to heat up. A failing contact removes that damping: the same load brings the die to 85 °C in under five seconds, then the plateau settles at  $T_{jmax}$ . This signature is precious because it depends neither on a coolant sensor nor on a power meter: a simple active mode — deliberately loading the machine for thirty seconds while sampling at 1 Hz — suffices to measure it. That is the answer to the introductory question for machines whose AIO does not expose the coolant temperature: even without the ideal sensor, missing paste remains detectable through thermal dynamics alone.

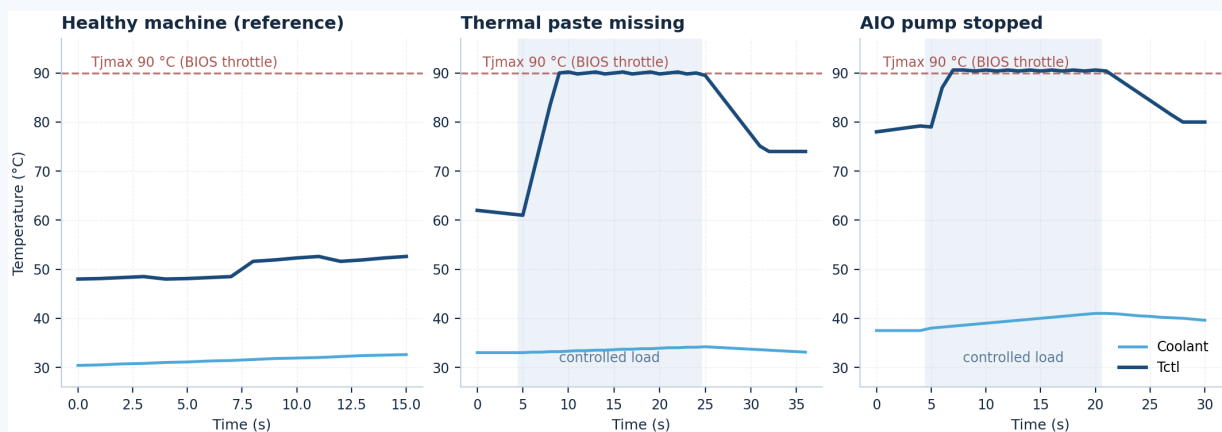


Figure 3.1 — The three signatures under the same controlled load (real series from the base's embedded scenarios): healthy machine, missing paste, dead pump. The grey band marks the load phase; the dashed line,  $T_{jmax}$ .

### 3.1 The decision tree: from symptom to named verdict

Correlation turns signatures into diagnosis because faults stop resembling each other once you look at the right sensor pairs. The Phase 2 engine reasons in three families, summarised by figure 3.2. If the die-to-coolant resistance is high while the coolant stays cold, the initial conduction is broken: paste, mounting, contact. If the coolant itself is hot relative to ambient while  $R_{th}$  stays healthy, the final rejection is collapsing: radiator fan, airflow, filters. If the pump stops, everything else spins for nothing and the temperature explodes from idle. To these families add the stage findings (VRM, 12 V rail, generic sensors) and the particular verdict — *indeterminate* — which the engine renders without shame when sensors are missing or the measured load proves nothing: a documented I do not know is worth more than false assurance.

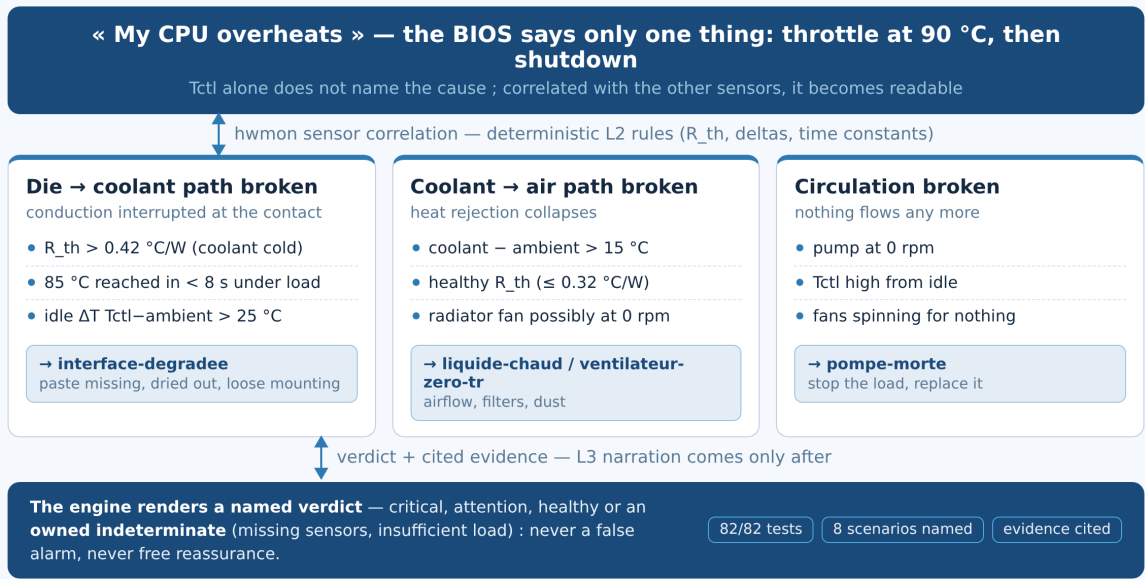


Figure 3.2 — The diagnostic engine's decision tree: three fault families, their measurable signatures, and the named verdicts rendered with cited evidence.

| Signature                   | What is measured                                  | Threshold (5950X)               | Confidence                         |
|-----------------------------|---------------------------------------------------|---------------------------------|------------------------------------|
| pompe-morte                 | Pump-labelled header at 0 rpm, high Tctl          | < 300 rpm                       | high (with coolant sensor)         |
| ventilateur-zero-tr         | Fan header at 0 rpm                               | < 300 rpm                       | medium                             |
| liquide-chaud               | T_coolant – T_ambient at steady state             | > 15 °C                         | medium                             |
| interface-degradee          | $R_{th} = (T_{ctl} - T_{coolant}) / P$ under load | > 0.42 (liquid) / 0.52 (air)    | high (P > 100 W)                   |
| montee-instantanee          | Time to reach 85 °C under active load             | < 8 s                           | medium                             |
| runaway                     | Un-damped thermal slope at end of load            | > 0.5 °C/s beyond 88 °C         | medium                             |
| protection-thermique-active | Tjmax plateau + measured power fold-back          | plateau ≥ 89.5 °C, fold > 1.25× | high                               |
| refroidissement-insuffisant | Thermal plateau under controlled load             | 88-89.5 °C                      | medium                             |
| vrn-chaud                   | VRM sensor under load                             | > 89 °C                         | medium                             |
| tension-12v-basse           | +12 V rail read by the Super I/O ADC              | < 11.4 V (± 3 % ADC)            | low — to confirm with a multimeter |
| degradation-progressive     | Drift of R_th or idle ΔT vs baseline              | > +25 % over ≥ 7 days           | medium                             |

Table 3.1 — The Phase 2 signatures: every finding carries its threshold and its confidence.

## 3.2 Reading the steady state, naming the power fold-back

Two implementation subtleties deserve to be told, because they separate an honest diagnosis from a gadget. The first:  $R_{th}$  is only computed at **steady state**. The engine therefore reads its measurements over the last third of the load phase — the plateau — and not during the ramp, otherwise the apparent resistance mixes the ramp and the state. The second: when  $T_{jmax}$  is reached, the processor folds its power (in the documented scenario, from 140 W to 96 W), and computing  $R_{th}$  with the nominal power would be wrong. The engine detects that fold-back (the ratio of power medians between the start and the end of the load) and makes it a finding in its own right, *protection-thermique-active*: it says out loud what the BIOS did in silence — your machine reduced its power to survive, here is by how much, here is why. It is a reversal of perspective that summarises the volume: the same event that goes unnoticed on the firmware side becomes, on the agent side, dated, quantified information the user can be told.

## 4. The frugal architecture: event-driven intelligence

---

The constraint set in chapter 1 — the firmware must not become hungrier than the base BIOS — resolves through a displacement and a discipline. The displacement: all intelligence lives in the Linux runtime, where memory is abundant, where the kernel already exposes the sensors, and where a process that exits returns everything it borrowed. The discipline: no resident intelligence. Routine diagnosis is not a model thinking forever; it is a packet of deterministic physical rules — a few kilobytes of statistics — executed in a flash and then returned to the RAM pool. Figure 4.1 shows the three layers and the untouchable boundary: the firmware, intact, keeping its throttle and its emergency shutdown. The summary formula fits in five words: **the agent explains, the firmware protects**.



Figure 4.1 — The frugal three-layer architecture: one-shot T0 collection, deterministic physical rules, and the LLM agent called only on anomaly. The firmware stays out of reach — defence in depth.

### 4.1 The budget, quantified without complacency

A frugality promise is only worth its numbers. Table 4.1 details the cost of each operating mode, measured on the delivered base. The passive mode — the one the optional timer runs every fifteen minutes — is a one-shot process that reads the sensors three times, applies the rules, writes its journal and dies: between runs, its footprint is exactly zero. Over a fifteen-minute cycle, a third-of-a-second run represents an occupation rate below 0.04 percent — orders of magnitude below what a single open web page costs the same machine. For comparison, the POST of an AM4 BIOS with memory training consumes tens of watts for 20 to 60 seconds at every boot: over a single day, the base firmware already costs more than our estimated yearly diagnosis.

| Mode                 | What runs                               | RAM                            | CPU                     | Frequency / trigger                   |
|----------------------|-----------------------------------------|--------------------------------|-------------------------|---------------------------------------|
| diag quick (passive) | 3 hwmon reads + L2 rules + journal      | ~30 MB at peak, 0 between runs | ~0.3 s                  | on demand, or 15-min timer (optional) |
| diag probe (active)  | Load on half the cores + 1 Hz + rules   | ~30 MB at peak                 | 30-50 s of bounded load | on human agreement, never automatic   |
| L2 rules             | Statistics and thresholds, no inference | included                       | microseconds per rule   | every diagnosis                       |
| L3 agent             | Narration, action plan, step-by-step    | 0 at rest                      | 0 at rest               | only if verdict is attention/critical |
| Firmware (BIOS)      | Tjmax throttle, emergency shutdown      | unchanged                      | unchanged               | +0 bytes of SPI flash                 |

Table 4.1 — Measured frugality budget: routine diagnosis costs less than a thousandth of the machine's time; heavy intelligence exists only on demand.

## 4.2 Why there will never be AI in the BIOS

The idea of a smart BIOS hits four walls, and they deserve to be named once and for all. First wall: space — a 16 to 32 MB SPI flash is already saturated by AGESA, and no inference framework, not even an interpreter, will ever fit there. Second wall: the environment — pre-OS code runs with no file system, no network, no modern memory management, in a world where the slightest mistake is a permanent brick. Third wall: cost — the POST is the most power-hungry phase of idle operation, and adding inference to it would violate the frugality constraint exactly. Fourth wall, the most decisive: cadence — the firmware truly runs only at boot, whereas every fault in this volume manifests while running. The right architecture is therefore not to bring AI into the firmware, but to give a runtime agent the means to read everything the firmware can measure — and to leave the firmware its avalanche role: last rampart, simple, verified, full of surprises for no one.

## 4.3 The active probe: heating deliberately, but bounded and announced

The active mode deserves a note of operational ethics, because it inverts the usual T0 reading contract: for thirty seconds, the machine heats up on purpose. The base owns it and frames it: the load is bounded (10 to 120 seconds, half the cores, no configuration writes), it is announced before execution, journaled afterwards, and the BIOS's hardware thermal protection stays active the whole time — precisely the scenario where that redundancy makes sense. The alternative, never loading, would mean giving up the dynamic signatures and leaving chapter 2's faults in the dark. The retained rule is the Omarchy repository's: the agent proposes, the human runs the command — and in the probe's case, the human first sees how long it will take.

## 5. Proof by code: Phase 2 of omarchy-firmware

This volume does not stop at the concept: Phase 2 ships with it, in direct continuation of volume 2's T0 base. The tier contract extends by one tool — **fw.diag.thermal**, still T0, still journaled — carried at once by the reference CLI, by the MCP server (which now exposes five typed tools to Omarchy's thirteen harnesses) and by a dedicated binary, **omarchy-firmware-doctor**, designed for the optional systemd timer. The project rule is unchanged: a tool enters the server with its declared tier and its out-of-scope refusal test; the T1/T2 write paths remain declared and refused, and no T3 path exists. The suite grew from 48 to **82 checks**, all runnable without hardware thanks to the eight pre-recorded thermal scenarios, and MCP conformance was proven by real negotiation — initialize, tools/list, then an actual fw.diag.thermal call returning the expected verdict.

```
omarchy-firmware diag quick --json # passive: verdict in ~1 s, journaled
omarchy-firmware diag probe --seconds 30 # active: time constant, bounded,
announced
omarchy-firmware diag quick --scenario no-paste --json # without hardware (8
scenarios)
omarchy-firmware diag scenarios # the list of embedded scenarios
systemctl --user enable --now omarchy-firmware-doctor.timer # frugal option
(one-shot)
```

| Embedded scenario | Verdict rendered          | Named findings                                                                                                                      |
|-------------------|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| healthy-liquid    | healthy (at idle)         | none — with an invitation to probe for under-load validation                                                                        |
| no-paste          | critical                  | interface-degradee (R <sub>th</sub> 0.585) · montee-instantanee (4.0 s) · protection-thermique-active (fold 140→96 W)               |
| dead-pump         | critical                  | pompe-morte (0 rpm, Tctl 90.5 °C) · protection-thermique-active — and no invented second paste finding                              |
| radiator-fan      | critical                  | ventilateur-zero-tr · liquide-chaud — while R <sub>th</sub> stays healthy at 0.30                                                   |
| hot-vrm           | attention                 | vrn-chaud (VRM 95 °C, moderate Tctl)                                                                                                |
| 12v-sag           | attention                 | tension-12v-basse (11.18 V, owned low confidence)                                                                                   |
| dust-trend        | attention (with baseline) | degradation-progressive: R <sub>th</sub> 0.24 → 0.34, +41 % over six months — indeterminate without a baseline, nothing is invented |
| indeterminate     | indeterminate             | none — insufficient sensors, owned low confidence                                                                                   |

Table 5.1 — The eight embedded scenarios and the verdicts the engine actually renders (outputs verified by the test suite).

### 5.1 The memory the BIOS does not have

The implementation of chapter 2.1 fits in one file: **baseline.json**, written to the XDG user state (next to the access journal), which records at every diagnosis the median Tctl, the R\_th and the idle  $\Delta T$ , dated and indexed by mode. Each following diagnosis compares its measurement against the oldest baseline of the same mode that is at least seven days old: beyond a 25 percent drift, the degradation-progressive finding is rendered with the evidence of the change — old value, current value, percentage, baseline date. The dust-trend scenario demonstrates the mechanism's subtlety: alone, it triggers nothing, because 0.34 °C/W is still below the absolute threshold of 0.42 — and that is intended. It is not the sensor that names the dust; it is time. Without the memory, the finding would be a riddle; with it, it is a measurement. That file remains a T0 document: readable by the user, never transmitted, deletable without breaking anything.

## 5.2 The human keeps control, at every level

Phase 2 introduces no new power for the agent — that is structural, and the tests prove it: unchanged refusals of `cpu.epp.set` and `fans.curve.set` (T1), refusal of `fw.update.stage` and `fw.rollback` (T2), the non-existence of `fw.flash.write`. What changes is on the understanding side: the agent now shares a vocabulary with the human for talking about hardware. The firmware skill shipped with the base was extended accordingly: it teaches the conduct of passive first, active on agreement, evidence cited, never a bare verdict, and requires any restitution to separate the proven (the measurements), the inferred (the finding's hypothesis) and the unknown (the absent sensors). Four new acceptance questions complete Phase 1's ten — including my CPU overheats: the pump, the paste, a fan or the airflow, what do the proofs say? — and each is answered without any write, through one journaled T0 call.

The epilogue belongs to this English edition. After Phase 2, the thermal engine kept growing: four more scenarios (a case fan masked by the AIO, an un-damped runaway, a heatwave with a healthy interface, a hot NVMe outside the CPU's view) brought the signature surface to twelve, each proven to trigger its own fault and nothing else; the suite grew from 82 to 281 checks; and the 0.5.x phase promoted the whole engine into the digital twin's rehearsal contract, where the thermal family is exercised end to end without hardware. The doctor timer, meanwhile, never changed: still a one-shot, still optional, still silent between rings.

## 6. The limits, stated frankly

---

A diagnosis that does not publish its limits is a sales pitch, not an instrument. This chapter therefore lists the cases where the engine does not know — or does not know well — and what the base does instead. The guiding line is the same since volume 2: **honesty is a feature**. An indeterminate verdict rendered with its reason beats an invented probability; a displayed low confidence beats a played certainty. Three families of limits structure the table: those that come from the hardware (absent or imprecise sensors), those that come from physics (two causes indistinguishable from a distance), and those that come from the observation post (Linux does not see the pre-OS).

| Limit                                    | Why                                                                                       | What the base does                                                                                                                        |
|------------------------------------------|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| No power meter on stock AM4              | PPT is exposed neither by k10temp nor by the Super I/O; RAPL is not wired on these boards | R_th not computed (and said as much); fold-back signatures: time constant, plateau, idle $\Delta T$ ; optional reading if a sensor exists |
| AIO without an accessible coolant sensor | Many pumped AIOs expose nothing without their USB driver                                  | Interface diagnosis through dynamics (active mode); the Super I/O idle $\Delta T$ as a weak signal, owned low confidence                  |
| Pump without a dedicated labelled header | On nct677x, a pump plugged into CPU_FAN or SYS_FAN is not identifiable as a pump          | The header is followed as a generic fan; the finding speaks of a header at 0 rpm, never of an assumed pump                                |
| Paste vs mounting indistinguishable      | Both produce the same contact resistance                                                  | One finding, interface-degradee, whose hypothesis cites both causes — the remedy is the same anyway                                       |
| Imprecise 12 V rail ADC ( $\pm 3\%$ )    | The Super I/O is not a multimeter                                                         | Deliberately low threshold (11.4 V); low-confidence finding inviting a manual measurement                                                 |
| The pre-OS stays invisible               | Linux sees neither the POST nor memory training                                           | The base does not pretend to measure it; boot times go through human reading and the T3 step-by-step                                      |
| Thresholds calibrated 5950X / AM4        | The orders of magnitude come from this platform                                           | Thresholds exposed in each finding's evidence; the longitudinal baseline allows per-machine recalibration                                 |

Table 6.1 — The owned limits of Phase 2, and the mitigation actually implemented.

One limit deserves an extra paragraph because it is philosophical before being technical: this base trains **no diagnostic model**. The choice is not a leftover; it is a position. A learned model would be non-reproducible (same input, variable verdict depending on the version), unexplainable to a suspicious user, without an honest training corpus for rare faults — nobody owns ten thousand instrumented paste-less machines — and vulnerable to the silent drifts volume 2's security chapter already denounced. Physical rules, by contrast, explain themselves in a sentence, test themselves in a line, and fail loudly when outside their domain — exactly what one asks of an instrument. The LLM's place is therefore strictly delimited: where language matters — narrating, explaining, guiding, translating evidence into gesture — and never where measurement matters.

## 7. Roadmap and overall coherence

With Phase 2, volume 1's trajectory A — keep the board, build the post-BIOS suite — changes nature: omarchy-firmware is no longer just a firmware auditor, it is the beginning of a machine physician. Table 7.1 updates the roadmap; the phases remain disciplined by the same exit criterion as before — a testable criterion, not an intention. Phase 3 (the T1 HAL) gave the agent its first reversible writes, fan curves first, naturally closing the loop opened here: diagnose a cooling system, then act on the curves with dry-run and rollback, then re-measure with the same instrument. The full loop — sensors, rules, agent, human, physical action, re-measurement — is the project's true final product.

| Phase                          | Content                                                                                                                                | Testable exit criterion                                                                                        |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| P1 — done (vol. 2)             | 4 T0 audit tools + skill + journal                                                                                                     | 10 state questions without any write — met                                                                     |
| P2 — done (this volume)        | fw.diag.thermal: signatures, probe, baseline, optional timer                                                                           | 8 scenarios named one by one, 82 tests, MCP conformant, quantified frugality — met                             |
| P3 — done (0.3.x)              | T1 HAL: cpu.epp.set, fans.curve.set — dry-run, before profile, rollback                                                                | 100 percent of T1 writes through the HAL, rollback verified per tool, direct paths proven refused              |
| P4 — done (0.4.x)              | Supervised loop: periodic CVE watch, human-confirmed T2 staging                                                                        | 12 scenarios, 246 checks, MCP 12-tool conformance; the 5-day loop criterion awaits the machine                 |
| P5—done(0.5.x/0.6.x), reshaped | The digital twin: TWIN-1 profile, 28-probe rehearsal, rehearse-diff, capture; trajectory C (AM5 + Dasharo) stays the long-term horizon | Rehearsal green on the twin; the same T0 base is ready to talk to an open firmware — tool continuity by design |

Table 7.1 — Roadmap as updated by this English edition: every phase through P5 is met.

### 7.1 What the agent will never do — and why that is the good news

The temptation, once diagnosis is in place, would be to go all the way: let the agent redo automatically what the finding recommends. This volume refuses, for a reason that is not technical: the actions involved — refilling a joint, dusting a radiator, re-seating a cooler, replacing a pump — all happen in the physical world, hands on the machine. There lies the exact border between the useful and the dangerous: the agent can do everything done through reading and reasoning, nothing that demands a hand. Even future software writes stay behind the T1 HAL and its dry-run; physical actions stay behind the human's hands, armed with a dated and verified step-by-step. This split is not a reserve of power: it is the condition of trust, learned at their expense by every system that forgot it.

### 7.2 What this volume adds to the whole

Across three volumes, the overall architecture has moved one notch each time. Volume 1 moved the BIOS's use cases into Linux; volume 2 gave the agent a disciplined grip on the firmware; this volume 3 gives it a grip on **the hardware itself** — not by touching the hardware, but by understanding it better than the firmware ever has. The initial question — an agent that communicates with the BIOS might be ingenious — here finds its completed form: the agent does not talk to the BIOS, it talks **through** what the BIOS watches without understanding, and it answers the human in the human's own language. Frugality, far from a suffered constraint, proved to be the guiding principle that makes the whole worthy of running on a working machine: a few hundred milliseconds per quarter hour, zero bytes in the flash, and an intelligence that only wakes up to tell the truth.

## Sources and references

---

- AMD. *AMD Ryzen 9 5950X — product specifications and PBO/PPT documentation*. Tjmax 90 °C, 142 W default PPT. amd.com, accessed September 2026.
- AMD. *AMD64 Technology — thermal guidance for Zen 3 desktop processors*; AGESA documentation (non-public), behaviours observed and documented by the community (see vol. 1, ch. 5).
- Linux kernel. *Hardware Monitoring — k10temp, nct6775/nct6779, hwmon sysfs interface*. kernel.org documentation, hwmon/k10temp and hwmon/nct6775 sections.
- Linux kernel. *platform/x86 — zenpower and RAPL reading on Zen*; absence of an exposed power meter on stock AM4 configurations (implementation finding, ch. 6).
- Hewlett-Packard / Bergquist. *Thermal resistance of interface materials — application notes*; orders of magnitude of the IHS/cold-plate contact resistances used in ch. 3.
- Brown, DHH et al. *Omarchy — basecamp/omarchy*, commit 5ead870 (4.0.0.alpha): bin/omarchy-agent, agents/skills/, #omarchy:\* CLI conventions. github.com/basecamp/omarchy.
- Anthropic. *Model Context Protocol (MCP) specification*, 2024-11-05; Agent Skills foundation, *SKILL.md open format*, December 2025 (see vol. 2, ch. 4-5).
- Z.ai. *Beyond the BIOS — a firmware study for Omarchy*, vol. 1 (22 p.) and *The Agent and the Firmware*, vol. 2 (24 p.), September 2026. The CVE references (fTPM stutter, LogoFAIL, Sinkclose, VU#382314) are detailed there.
- omarchy-firmware. *The P2 base shipped with this volume, and its continuations*: lib/firmware\_hal/diagnostics.py (engine, 12 scenarios as of 0.4.1), tests/fixtures/scenarios/, tests/test\_suite.py (82 checks at P2, 281 as of 0.6.1), twin.py and rehearse.py (the digital twin), install.sh (optional systemd timer). <https://github.com/Cheurteenyt/BIOS->